Workshop 1: Introduction to scikit-learn

TECH6

Richard Foltyn
NHH Norwegian School of Economics

October 15, 2026

See GitHub repository for notebooks and data:

https://github.com/richardfoltyn/TECH6-H26

Contents

1 Exercise 1: Noise and estimation precision **1**

2 Exercise 2: Polynomial under- and overfitting **3**

Exercise 1: Noise and estimation precision

We revisit the lecture's simple linear model:

$$y_i = \alpha + \beta x_i + \epsilon_i, \qquad \alpha = 1, \quad \beta = 0.5, \qquad \epsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2).$$

The explanatory variable x_i is uniformly distributed on $[0, 10]$ and independent of the noise ϵ_i. How do the noise standard deviation σ and sample size N affect the estimated intercept and slope?

The sample-generation function below is taken from the lecture. A plotting helper is also provided which you can use to visualize the sample, the true relationship, and the estimated regression line. Run their cells before starting the exercise.

[1]:
```
def create_sample(N, alpha, beta, sigma):
    """
    Create sample for simple linear regression

    Parameters
    ----------
    N : int
        Sample size
    alpha : float
        Intercept (or bias)
    beta : float
        Slope (or weight)
    sigma : float
        Standard deviation of the error term

    Returns
    -------
    x : array
        Predictor variable, uniformly distributed on [0, 10]
    y : array
        Outcome variable
```

```
    """
    from numpy.random import default_rng

    # Initialize random number generator with a fixed seed
    rng = default_rng(seed=1234)

    # Use x that are uniformly distributed on [0, 10]
    x = rng.uniform(0, 10, size=N)

    # Normally distributed errors
    epsilon = rng.normal(scale=sigma, size=N)

    # Create outcome variable
    y = alpha + beta * x + epsilon

    return x, y
```

The plotting helper takes the sample arrays `x` and `y`, a fitted model, the true intercept `alpha` and slope `beta`, and a title. It plots the sample with the true and estimated lines.

[2]:
```
import matplotlib.pyplot as plt
import numpy as np


def plot_sample(x, y, model, alpha, beta, title, ylim=None):
    """
    Plot a regression sample with the true and estimated lines.

    Parameters
    ----------
    x : array
        Explanatory variable
    y : array
        Outcome variable
    model : object
        Fitted linear regression model
    alpha : float
        True intercept
    beta : float
        True slope
    title : str
        Plot title
    ylim : tuple, optional
        y-axis limits (ymin, ymax)
    """
    _fig, ax = plt.subplots(figsize=(5, 3.5), layout='constrained')
    x_grid = np.linspace(0, 10, 101)
    y_true = alpha + beta * x_grid
    X_grid = x_grid[:, None]
    y_hat = model.predict(X_grid)

    ax.scatter(x, y, s=15, c='steelblue', alpha=0.5, label='Sample')
    ax.plot(x_grid, y_true, c='black', lw=1.5, ls='--', label='True line')
    ax.plot(x_grid, y_hat, c='darkorange', lw=1.5, label='Estimated line')

    # Add text with alpha and beta estimates in the lower right corner of the plot
    ax.text(
        0.95,
        0.05,
        rf'$\widehat{{\alpha}} = {model.intercept_:.3f}, \widehat{{\beta}} = {model.
↪coef_[0]:.3f}$',
        transform=ax.transAxes,
        ha='right',
```

```
        va='bottom',
    )
    # Add remaining annotations
    ax.set_title(title)
    ax.set_xlabel('$x$')
    ax.set_ylabel('$y$')
    ax.set_xlim(0, 10)
    if ylim is not None:
        ax.set_ylim(ylim)
    ax.legend()
```

Part 1 — Increasing the noise

First, you are going to investigate how the estimated model coefficients change as the noise standard deviation σ increases.

Fix $N = 50$, $\alpha = 1$, and $\beta = 0.5$. Write a loop over $\sigma \in \{0.2, 0.7, 2.0\}$. For each noise standard deviation:

1. Generate a sample using `create_sample()`. The fixed seed keeps the x values and underlying standard-normal noise draws identical across noise levels; only the noise scale changes.
2. Fit a `LinearRegression()` model, using `X = x[:, None]` as the feature matrix.
3. Print `model.intercept_` and `model.coef_[0]`, and compare them with the true values.
4. Call `plot_sample()` to visualize the sample, the true line, and the estimated line.

How does increasing σ affect the scatter around the true line and the coefficient estimates?

Part 2 — Increasing the sample size

Fix $\sigma = 2.0$ and compare samples of $N = 50$ and $N = 1{,}000$, keeping $\alpha = 1$ and $\beta = 0.5$. For each sample size:

1. Draw a sample using `create_sample()`.
2. Fit each model and print its estimated intercept and slope.
3. Call `plot_sample()` to visualize the sample, the true line, and the estimated line.

Are the estimates closer to the true values in the larger sample? Explain why a larger sample improves coefficient precision without reducing the underlying noise.

Exercise 2: Polynomial under- and overfitting

Consider the following non-linear model,

$$y_i = f(x_i) + \epsilon_i$$
$$f(x) = \cos\left(\frac{3}{2}\pi x\right)$$
$$\epsilon_i \overset{\text{iid}}{\sim} N(0, \sigma_\epsilon^2)$$

where y_i is a trigonometric function of x_i but is measured with an additive error ϵ_i. In this exercise, we are going to approximate y_i using polynomials in x_i of varying degrees.

Part 1 — Creating and plotting a sample

The function `fcn()` implements the true relationship $y = f(x)$.

[5]:
```python
import numpy as np


def fcn(x):
    """
    True function without errors.
    """
    return np.cos(1.5 * np.pi * x)
```

Using `fcn()`, create a sample of size $N = 50$. Draw the x_i from a uniform distribution on $[-1, 1]$ and use $\sigma_\epsilon = 0.2$. Initialize your RNG with seed 123, then generate y_i according to the model above.

Create a scatter plot of the sample (x_i, y_i) and add a line showing the true relationship without measurement error.

Part 2 — Polynomial approximations

First, convert the sample's explanatory variable to a feature matrix using `X = x[:, None]`.

Complete the template for `plot_poly_degrees()` below to create a figure with 6 subplots, showing fitted polynomials of degrees $d \in \{0, 1, 2, 4, 6, 12\}$. Each panel should also show the sample scatter plot and the true function $y = f(x)$.

How does the approximation change as you increase d? Do higher-degree polynomials always predict new observations better?

Hints:

- Build a pipeline, for example using `make_pipeline()` to combine the `PolynomialFeatures()` transformation and `LinearRegression()` to approximate y as a polynomial in x.
- When creating polynomials with `PolynomialFeatures(..., include_bias=True)`, use `LinearRegression(fit_intercept=False)`. The polynomial transformation already includes the constant term.

[8]:
```python
import matplotlib.pyplot as plt


def plot_poly_degrees(degrees, X, y):
    """
    Fit and plot polynomial regression models of different degrees.

    Parameters
    ----------
    degrees : array-like
        List of polynomial degrees to fit and plot.
    X : array-like
        Explanatory variable.
    y : array-like
        Response variable.
    """

    # Create figure with 3 columns
    ncol = 3
    nrow = int(np.ceil(len(degrees) / ncol))

    _fig, axes = plt.subplots(
        nrow, ncol, figsize=(8, 5), sharex=True, sharey=True, layout='constrained'
    )

    # x-values for predicting & plotting
    xvalues = np.linspace(-1.0, 1.0, 201)
    # True y-values
```

```
    y_true = fcn(xvalues)

    for i, ax in enumerate(axes.flatten()):
        # Degree for current sub-plot
        d = degrees[i]

        # TODO: Create polynomial transformation + estimation pipeline

        # TODO: Fit the model to the sample data

        # TODO: Predict the response variable for the x-values

        # TODO: Plot predicted values

        # Plot true relationship
        ax.plot(xvalues, y_true, c='black', lw=1.0)
        # Plot sample as scatter plot
        ax.scatter(X, y, s=20, c='none', ec='steelblue', lw=0.75)
        ax.set_ylim((-1.5, 1.5))
        ax.legend()
```

Part 3 — Training and validation errors

Evaluate how well polynomials of degrees $d = 0, 1, \ldots, 12$ fit the training data and predict unseen observations.

Use `train_test_split()` from `sklearn.model_selection` to split `(X, y)` into training (70%) and validation (30%) subsets, setting `random_state=123`.

The validation set helps choose the polynomial degree. A separate test set is reserved for evaluating prediction error after that choice is fixed. We do not use a test set in this exercise.

Next, complete the template for `compute_poly_mse()` below. The function takes a single polynomial degree `d` and the training and validation data. It should:

1. Build a pipeline combining `PolynomialFeatures(degree=d, include_bias=True)` and `LinearRegression(fit_intercept=False)`.
2. Fit the pipeline on the training data `(X_train, y_train)`.
3. Predict the response on both the training and validation sets.
4. Compute the mean squared error (MSE) on each subset using `mean_squared_error()`.

Return the two MSEs as `mse_train, mse_val`.

[11]:
```
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# TODO: Split sample into training (70%) and validation (30%) sets
# X_train, X_val, y_train, y_val = ...


def compute_poly_mse(d, X_train, y_train, X_val, y_val):
    """
    Compute training and validation MSEs for one polynomial degree.

    Parameters
    ----------
    d : int
        Polynomial degree
    X_train : array
        Training feature matrix
```

```
    y_train : array
        Training outcomes
    X_val : array
        Validation feature matrix
    y_val : array
        Validation outcomes

    Returns
    -------
    mse_train : float
        Training MSE
    mse_val : float
        Validation MSE
    """
    # TODO: Build pipeline
    # TODO: Fit on training data
    # TODO: Predict on training and validation data
    # TODO: Compute and return the two MSEs
```

Part 4 — Optimal polynomial degree and error curves

Iterate over polynomial degrees $d = 0, 1, \ldots, 12$. For each degree, call `compute_poly_mse()` and save the training and validation MSEs.

Plot the training MSE and validation MSE against d. Use `np.argmin()` to determine which degree gives the lowest validation MSE.

How do the training and validation errors change as the degree increases? Explain why the model with the lowest training MSE may predict new observations poorly.

Part 5 — Plotting the fitted model

Re-estimate the model on the training set using the degree selected in Part 4. Create a plot showing:

- The true relationship $y = f(x)$
- The training observations
- The validation observations, using a distinct marker and color
- The fitted polynomial curve evaluated on a dense grid $x \in [-1, 1]$